Especificación Técnica: Arquitectura y Soporte Multi-Cámara (Dual RealSense)

1. Introducción y Justificación de Ingeniería

El escaneo 3D monocular (con una sola cámara) presenta limitaciones físicas insalvables: 1. **Oclusiones geométricas:** Toda superficie no visible directamente desde el plano focal de la cámara queda oculta ("sombras" de profundidad). 2. **Dependencia de movimiento:** Para escanear un objeto completo con una cámara, o bien el operador debe mover la cámara alrededor del sujeto, o bien el sujeto debe rotar en una plataforma giratoria, introduciendo desfases temporales y deformaciones en objetos no rígidos o sujetos vivos.

La integración de **dos cámaras Intel RealSense (D415 / D435 / D455)** de forma simultánea permite: * Captura de 180° a 360° en un único instante de tiempo (snapshot instantáneo sin desalineación por movimiento). * Eliminación de oclusiones cruzadas (p. ej. cara anterior y posterior, o cobertura lateral extendida). * Duplicación de la densidad de puntos sobre el volumen de interés.

Este documento establece las directrices de hardware, ancho de banda, sincronización y arquitectura de software para operar dos sensores concurrentes en la plataforma `RealSenseScans`.

---

2. Topología de Hardware y Gestión de Ancho de Banda USB

A. Límites del Bus USB 3.0 / 3.2 xHCI

Cada stream de la Intel RealSense opera transmitiendo dos flujos masivos de datos sin comprimir directamente al host: * **Profundidad:** Formato `RS2_FORMAT_Z16` (16 bits = 2 bytes por píxel). * **Color:** Formato `RS2_FORMAT_RGB8` (24 bits = 3 bytes por píxel).

Tabla de Consumo de Ancho de Banda Teórico por Cámara

Resolución	FPS	Flujo Profundidad (Z16)	Flujo Color (RGB8)	Total por Cámara	Total 2 Cámaras	Estado en USB 3.0 (5 Gbps teórico)
1280x720	30	~442 Mbps (55 MB/s)	~663 Mbps (83 MB/s)	**~1.10 Gbps**	**~2.21 Gbps**	Saturación frecuente en un solo host controller

Resolución	FPS	Flujo Profundidad (Z16)	Flujo Color (RGB8)	Total por Cámara	Total 2 Cámaras	Estado en USB 3.0 (5 Gbps teórico)
848x480 (Recomendado)	30	~195 Mbps (24 MB/s)	~293 Mbps (37 MB/s)	**~488 Mbps**	**~0.98 Gbps**	Estable y óptimo para streaming y captura
640x480	30	~147 Mbps (18 MB/s)	~221 Mbps (28 MB/s)	**~368 Mbps**	**~0.74 Gbps**	Máxima estabilidad en puertos compartidos

> [!IMPORTANT] Aunque USB 3.0 especifica 5 Gbps brutos, el ancho de banda efectivo utilizable por el protocolo isócrono/bulk xHCI ronda los 3.2 Gbps compartidos por todos los puertos del mismo controlador. Dos cámaras a 720p@30fps en el mismo controlador causan caídas de paquetes (`RuntimeError: Frame didn't arrive within 5000ms`).

B. Distribución Física Recomendada

1. **Controladores USB Separados:**
2. Cámara 1 conectada al panel posterior de la placa base (controlador xHCI principal de la CPU).
3. Cámara 2 conectada a una tarjeta de expansión PCIe USB 3.0 dedicada o a un bus secundario (controlador del Chipset).
4. **Comprobación en Windows (PowerShell):** Para verificar si las cámaras están en puertos o buses independientes: `powershell Get-PnpDevice -PresentOnly | Where-Object { $_.FriendlyName -like "*RealSense*" } | Select-Object FriendlyName, InstanceId`

---

3. Sincronización Temporal: Hardware Genlock vs. Software Sync

A. Sincronización por Hardware (Genlock Externo)

Para sujetos en movimiento, personas o piezas que vibran, la sincronización por hardware es la opción obligatoria para evitar artefactos temporales.

Esquema del Conector de Sincronización Externa (Pines traseros D400):

En la parte trasera o lateral de la cámara RealSense existe un conector plano de 8/9 pines: * **Pin 1:** Señal de sincronización (`SYNC_OUT` en Master / `SYNC_IN` en Slave) - Señal lógica TTL 1.8V. * **Pin 2:** Tierra común (`GND`).

```
+-------------------+                           +-------------------+
|  Cámara 1 (MASTER)|                           |  Cámara 2 (SLAVE) |
|  Pin 1: SYNC_OUT  |-------------------------->|  Pin 1: SYNC_IN   |
|  Pin 2: GND       |<------------------------->|  Pin 2: GND       |
+-------------------+                           +-------------------+
```

Configuración en `pyrealsense2` :

```
# Cámara 1: Configurar como Master (genera pulso en el inicio de la exposición)
depth_sensor_1 = profile_1.get_device().first_depth_sensor()
depth_sensor_1.set_option(rs.option.inter_cam_sync_mode, 1)

# Cámara 2: Configurar como Slave (espera pulso de sincronización)
depth_sensor_2 = profile_2.get_device().first_depth_sensor()
depth_sensor_2.set_option(rs.option.inter_cam_sync_mode, 2)
```

- **Comportamiento:** La cámara Slave frena su temporizador interno de lectura y sólo dispara el obturador del sensor estéreo en el instante exacto en que la cámara Master envía el pulso TTL, garantizando alineación a nivel de microsegundo ($\Delta t < 50\ \mu\text{s}$).

B. Sincronización por Software (Tolerante sin Cableado Físico)

Si no se dispone del cable físico de sincronización y el objeto a escanear es rígido y estático (montado en mesa o soporte fijo): 1. Cada cámara corre en su propio hilo trabajador y etiqueta cada fotograma con el reloj de hardware `frame.get_timestamp()` (en milisegundos). 2. Un gestor de emparejamiento por ventana de tiempo (`TimeSyncBuffer`) mantiene una cola circular y extrae pares de cuadros cuyo delta de tiempo cumpla: $$\Delta t = |t_{cam1} - t_{cam2}| \le \frac{1}{2 \cdot \text{FPS}} \approx 16.6\ \text{ms}\ (\text{a } 30\ \text{FPS})$$ 3. Si un cuadro queda huérfano por caída de tasa de refresco, se descarta para evitar desincronizaciones en la reconstrucción 3D.

---

4. Mitigación de la Interferencia de Proyectores Infrarrojos (IR)

Los modelos D415/D435/D455 integran un proyector láser infrarrojo que dibuja un patrón de puntos invisibles para dotar de textura artificial a superficies planas (paredes, metal liso, plástico blanco).

- **El Problema:** Si las dos cámaras apuntan a la misma región, los puntos del láser de la Cámara 1 se mezclan con los de la Cámara 2. Dado que el algoritmo estéreo busca calcular correspondencias binoculares, los dos patrones solapados pueden crear artefactos de ruido de profundidad.

- **Soluciones recomendadas en software:**
- **Modo Emitter Off en Cámara Secundaria:** `python # Dejar activo el emisor sólo en la cámara principal depth_sensor_slave.set_option(rs.option.emitter_enabled, 0)` La cámara Slave aprovecha el patrón de puntos proyectado por la cámara Master como si fuera textura ambiental, funcionando con total nitidez.
- **Emisor Alternado (Interleaved Sync):** Si se utiliza hardware sync, el modo `inter_cam_sync_mode = 3` permite que las cámaras alternen la iluminación cuadro a cuadro.

---

5. Diseño Arquitectónico de Software: `DualCameraEngine`

Para no bloquear el bucle de eventos del servidor asíncrono (`live3d/server.py`), la arquitectura multi-cámara se organiza en **dos hilos trabajadores independientes y un coordinador de sincronización**.

```
                          rs.context()
                               |
            +------------------+------------------+
            |                                     |
   CameraWorker (Master)                 CameraWorker (Slave)
   - Pipeline dedicado                   - Pipeline dedicado
   - Serial "11223344"                   - Serial "55667788"
   - Modo Sync: Master (1)               - Modo Sync: Slave (2)
   - Cola thread-safe                    - Cola thread-safe
            |                                     |
            +------------------+------------------+
                               |
                  DualFrameSyncCoordinator
                  - Emparejamiento por timestamp
                  - Watchdog individualizado
                  - Snapshot atómico: {cam1, cam2}
                               |
            +------------------+------------------+
            |                                     |
     Live 3D Stream Engine                Multi-Scan 3D Engine
     (WebGL Three.js GPU)                 (Open3D Fusión Dual)
```

Características de Robustez del Worker Dual:

1. **Descubrimiento por Número de Serie:** Las cámaras nunca se direccionan por índice (`[0]`), sino por su `serial_number` único garantizado por Intel.
2. **Aislamiento de Fallos:** Si una cámara sufre un microcorte USB, su worker entra en estado de recuperación sin tumbar la transmisión de la otra cámara.
3. **Arranque en Cascada:** Primero se inicializa la cámara Slave (para que esté lista en modo escucha) y finalmente la cámara Master (que empieza a emitir pulsos).

---

6. Estado Actual de Implementación y Validación en Hardware Real

A. Hardware Físico Conectado y Verificado

- **Cámara 1 (Master):** Intel RealSense D455 · Serial `207322250777` · Firmware `5.17.3.10` · Conexión USB 3.2 (SuperSpeed).
- **Cámara 2 (Slave):** Intel RealSense D455 · Serial `203522251709` · Firmware `5.12.7.150` · Conexión USB 3.2 (SuperSpeed).

B. Métricas de Rendimiento en Tiempo Real

- **Streaming Simultáneo:** Cam 1 a **30.0 FPS** y Cam 2 a **30.2 FPS** estables.
- **Tasa de Caída de Cuadros:** 0% de pérdidas de tramas bajo bus USB 3.2.
- **Tiempo de Respuesta en Captura Dual:** < 150 ms para sincronizar y transferir cuadros de ambas cámaras a Open3D.

C. Hallazgos Críticos de Integración y Soluciones Implementadas en el Código

1. **Sincronización Hardware vs. Software:**
2. La opción `inter_cam_sync_mode` de librealsense genera excepción crítica si se intenta modificar en caliente con el pipeline activo.
3. Si se activa el modo Slave (`inter_cam_sync_mode = 2`) sin tener soldados físicamente los 2 hilos del conector GPIO (Pines 1 y 2 a 1.8V TTL), la cámara detiene su exposición esperando un pulso eléctrico externo que nunca llega, causando timeout.
4. **Solución implementada:** Se opera con `sync_mode = 0` (software sync transparente) utilizando el timestamp de hardware (`frame.get_timestamp()`), lo que permite 30 FPS continuos en cualquier PC sin necesidad de modificaciones electrónicas.
5. **Mitigación Óptica de Interferencia IR:**
6. Implementado en `DualRealSenseSource`: la Cámara 1 opera con proyector encendido (`emitter_enabled = 1`) y la Cámara 2 con emisor apagado (`emitter_enabled = 0`). La Cámara 2 aprovecha el patrón de textura de la Cámara 1, eliminando el ruido de solape.
7. **Gestión de Recursos USB en Windows:**
8. Se incorporó la rutina de reseteo por hardware (`d.hardware_reset()`) y limpieza de procesos en segundo plano para garantizar que los endpoints USB queden completamente liberados tras cada sesión.